

Tut 11

Anne

Was sind WSDL & SOAP?

WSDL

Web Services Description Language

- **Der Vertrag**
- Beschreibt den Service in XML:
 - Welche Methoden gibt es?
 - Welche Parameter?
 - Welche Rückgabetypen?
 - Wohin sende ich den Request?
- **Client liest WSDL -> weiß alles**

SOAP

Simple Object Access Protocol

- **Das Nachrichtenformat**
- XML-basiertes Protokoll für Kommunikation:
 - Immer über HTTP
 - Inhalt = XML im Body
- Feste Struktur: Envelope -> Body



WSDL sagt WAS, SOAP sagt WIE

Beispiel: WSDL eines Wetterdienstes

Methode: public float GetCurrentTemperature(String city, String unit){}

```
<s:element name="GetCurrentTemperature">
  <s:complexType>
    <s:sequence>
      <s:element name="city" type="s:string"/>
      <s:element name="unit" type="s:string"/>
    </s:sequence>
  </s:complexType>
</s:element>

<s:element name="GetCurrentTemperatureResponse">
  <s:complexType>
    <s:sequence>
      <s:element name="GetCurrentTemperatureResult" type="s:float"/>
    </s:sequence>
  </s:complexType>
</s:element>

<wsdl:operation name="GetCurrentTemperature">
```

SOAP Request

HTTP-Header

```
POST /Pfad HTTP/1.1
Host: server.de
Content-Type: text/xml
SOAPAction: "..."
```

XML Body: Envelope

<soap:Body>

Methodenaufruf
mit Parametern

```
POST /WeatherService.asmx HTTP/1.1
Host: api.wetter.de
Content-Type: text/xml; charset=utf-8
SOAPAction: ".../GetCurrentTemperature"

<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope
  xmlns:xsi="...XMLSchema-instance"
  xmlns:xsd="...XMLSchema"
  xmlns:soap="...soap/envelope/">
  <soap:Body>
    <GetCurrentTemperature
      xmlns="http://api.wetter.de/">
      <city>Berlin</city>
      <unit>Celsius</unit>
    </GetCurrentTemperature>
  </soap:Body>
</soap:Envelope>
```

SOAP Response

Der Server antwortet immer mit 200 OK + gleichem Envelope-Gerüst

REQUEST (Client -> Server)

```
<soap:Body>
  <GetCurrentTemperature
    xmlns="http://api.wetter.de/">
    <city>Berlin</city>
    <unit>Celsius</unit>
  </GetCurrentTemperature>
</soap:Body>
```



RESPONSE (Server → Client)

```
HTTP/1.1 200 OK

<soap:Body>
  <GetCurrentTemperatureResponse
    xmlns="http://api.wetter.de/">
    <GetCurrentTemperatureResult>
      <float>18.5</float>
    </GetCurrentTemperatureResult>
  </GetCurrentTemperatureResponse>
</soap:Body>
```

SOAP Response

Der Server antwortet immer mit 200 OK + gleichem Envelope-Gerüst

REQUEST (Client -> Server)

```
<soap:Body>
  <GetPositionOfSymbolInText
    xmlns="http://tu-berlin.de/3s/anwsys/"
    <text>Anwendungssysteme ...</text>
    <symbol>a</symbol>
  </GetPositionOfSymbolInText>
</soap:Body>
```



RESPONSE (Server → Client)

```
HTTP/1.1 200 OK

<soap:Body>
  <GetPositionOfSymbolInTextResponse
    xmlns=" http://tu-berlin.de/3s/anwsys/">
    <GetPositionOfSymbolInTextResult>
      <int>1</int>
      <int>18</int>
    </GetPositionOfSymbolInTextResult>
  </ GetPositionOfSymbolInTextResponse >
</soap:Body>
```

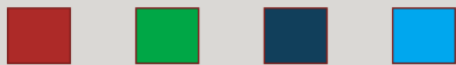
Microservices

- **Unabhängig deploybar** -> Service updaten ohne die anderen zu berühren
- **Eigene Datenbank**
- jeder Service darf andere Sprache/Technologie nutzen
- **Choreographie** -> kommunizieren direkt miteinander (kein zentraler Orchestrator)
- **DevOps-Team** -> ein Team besitzt und betreibt einen Service komplett
- **Nachteil:** Schwer zu überblicken, viele Netzwerk hops, komplexes Debugging

Login, Zahlung, Produkte zusammen

Architecture before microservices

Presentation layer



Business layer

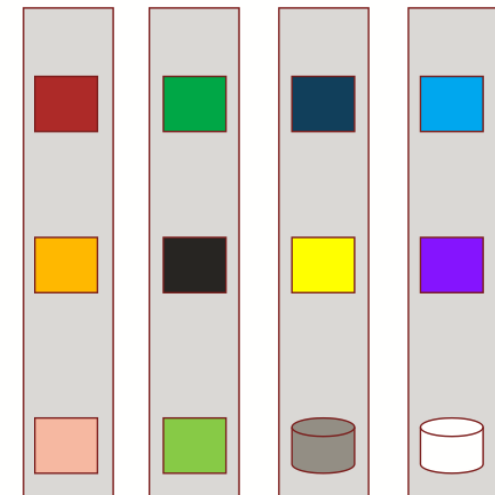


Resource layer



Login Service, Zahlung Service, Produkt Service, Bestell Service

Microservice-based architecture



Orchestrierung vs. Choreographie

Orchestration

Zentraler Orchestrator kennt den gesamten Ablauf und ruft einzelne Services auf.

Vorteile:

- Zentrales Prozesswissen
- Leicht zu konzipieren

Nachteile:

- Single Point of Failure
- Performance-Bottleneck
- Doppelte Netzwerk hops (double billing!)

VS

Choreographie

Dezentrales Wissen -> jeder Service weiß selbst, wie er reagiert.

Vorteile:

- Keine extra Hops -> niedrige Latenz
- Kein zentraler Bottleneck

Nachteile:

- Schwer zu überblicken
- Kann chaotisch wirken

SQS

Simple Queue Service

- Simple: nur 3 Operationen: Send Message, Receive Message, Delete Message
- -> Einseitig

Amazon Web Services bietet einen cloudbasierten Messaging-Dienst an. Der „Simple Queue Service“ (SQS) verfügt über folgende Operationen: Send Message, Receive Message, Delete Message.

a) Beschreiben Sie die typische Interaktion zwischen Client und Server über SQS-Queues, wenn der Client ein Ergebnis zurückbekommen soll.

1. Client schickt Request an Queue Q1
2. Server empfängt Nachricht von Q1 und verarbeitet diese
3. Server packt die Antwort in Queue Q2
4. Client empfängt Nachricht von Q2 (und ordnet diese über Felder in der Nachricht, **z. B. eine ID**, einer Anfrage zu)

SQS

Simple Queue Service

- Simple: nur 3 Operationen: Send Message, Receive Message, Delete Message
- -> Einseitig

b) Stellen Sie sich vor, Sie sind Software Architect in einem Unternehmen, in dem Microservices eingesetzt werden. Würden Sie SQS benutzen, um die Kommunikation zwischen allen Services ausfallsicherer zu machen? Nennen Sie Vor- und Nachteile.

1. **Services werden entkoppelt** -> durch Redundanz in der Queue können Übertragungsfehler vermieden und Nachrichten garantiert zugestellt werden
2. **Loose Coupling:** Server und Client können unabhängig voneinander starten/stoppen/skalieren
3. **Loose Coupling II:** Das Interface zwischen den Komponenten ist standardisiert und kann aus unterschiedlichen Programmiersprachen benutzt werden

SQS

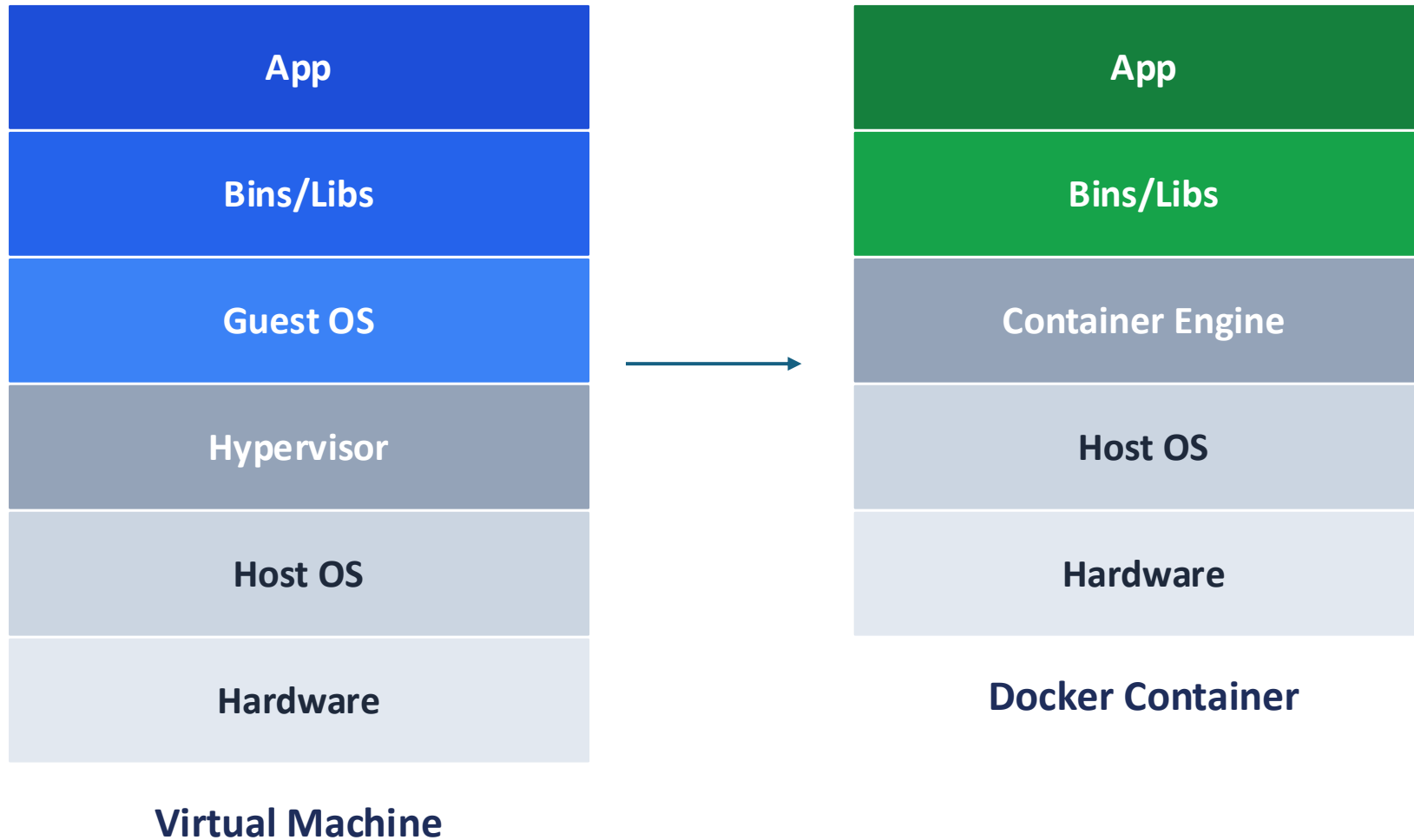
Simple Queue Service

- Simple: nur 3 Operationen: Send Message, Receive Message, Delete Message
- -> Einseitig

b) Stellen Sie sich vor, Sie sind Software Architect in einem Unternehmen, in dem Microservices eingesetzt werden. Würden Sie SQS benutzen, um die Kommunikation zwischen allen Services ausfallsicherer zu machen? Nennen Sie Vor- und Nachteile.

1. Fehlende Cohesiveness (Smart Endpoints and Dumb Pipes): Durch ein sehr einfaches Interface zwischen (potenziell komplexen) Services geht Mächtigkeit verloren
 1. z. B. können mit SQS keine Prioritäten umgesetzt werden
 2. Außerdem muss gesamte Geschäftslogik auf dieses Pattern angepasst werden
2. Vendor Lock-In: Das Wechseln des Anbieters ist mit hohen Kosten verbunden.

VMs vs. Docker Container



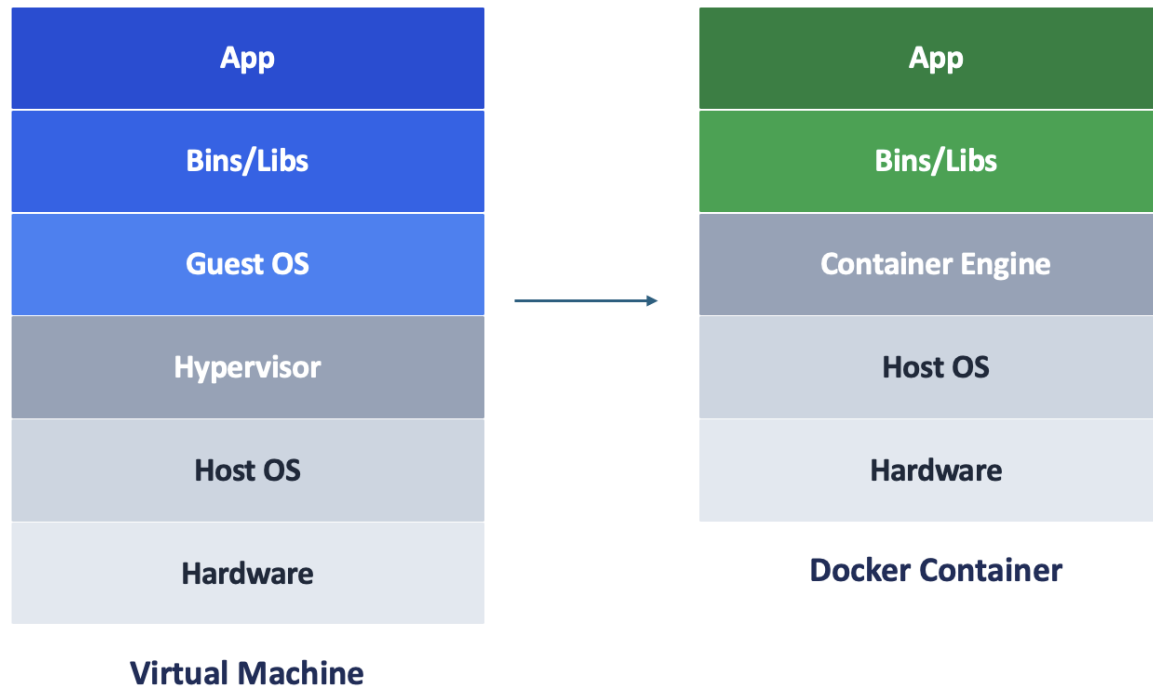
Docker Aufbau

Image = Schablone

- Technologie die wir brauchen
- Runtimes
- Tools/instructions um code zu runnen

Container = laufende Instanz eines Images

- docker run -> startet Container aus Image
- Isoliert aber leichtgewichtig
- Laufen abgeschirmt(eigene Libs, eigene Prozesse)
 - teilt Kernel des Host-OS
- Kurzlebig by design



Aufgabe:

Ein Startup möchte seine neue Web-App deployen. Der CTO schlägt vor, die App direkt auf einem dedizierten Server zu installieren. Ein Entwickler schlägt stattdessen Docker vor.

a) Nennen Sie zwei Vorteile von Docker gegenüber der direkten Installation auf dem Server.

Ihr Team entwickelt zwei Services: Service A benötigt Python 2.7 und Service B benötigt Python 3.11. Beide sollen auf demselben Server laufen.

a) Welches Problem entsteht bei direkter Installation auf dem Server?

b) Wie löst Docker dieses Problem?

GitHUb –
Code kommt
später

